35th Annual INCOSE international symposium
hybrid event
Ottawa, Canada
July 26 - 31, 2025

THE COST OF EXPERTISE: PERFORMANCE TRADE-OFFS IN LLMs FOR SYSTEMS ENGINEERING

Paul Wach
Virginia Tech National Security Institute
Virginia Tech Grado Department of Industrial & Systems Engineering
1311 Research Center Dr
Blacksburg, VA 24060, USA
+1(540)232-2127
paulw86@vt.edu

Ryan Bell
Naval Postgraduate School
1 University Circle
Monterey, CA 93943
ryan.bell@nps.edu

Brady Jugan
Virginia Tech National Security Institute
1311 Research Center Dr
Blacksburg, VA 24060, USA
bradyj66@vt.edu

Ryan Longshore
Naval Postgraduate School
1 University Circle
Monterey, CA 93943
ryan.longshore@nps.edu

Raymond Madachy
Naval Postgraduate School
1 University Circle
Monterey, CA 93943
rjmadach@nps.edu

Abstract. In the systems engineering (SE) community, generative artificial intelligence (GenAI), such as large language models (LLMs), continues to grow in popularity with applications varying from help with writing requirements, to creating traditional text-based documents, and to generating models. Furthermore, the text-based format of the Systems Modeling Language version 2 (SysMLv2) gives rise to wide ranging possibilities when combined with LLMs. It is common to specialize LLMs on specific tasks, such as SE; which may be achieved by a number of means. Fine-tuning and retrieval augmented generation (RAG) are example methods to teach a model how to act and what the model needs to know, respectively. Typical practice evaluates LLMs relative to one-another through the use of benchmarks. Benchmarks are tasks that use a common scoring-based method where the LLM either answers a set of multiple-choice questions or textual queries to verified, correct answers. A domain specific benchmark for SE, SysEngBench, was developed over the last year. With many in the SE community developing custom language models, the usage of a standard benchmark is essential for relative performance comparison. We have conducted a limited experiment using SysEngBench to compare performance, on SE tasks, of a small set of LLMs (control plus two modified models). The intent of this study was to set a baseline for ranking models according to performance on SE tasks, the results suggest what many in the SE community may find intuitively unexpected.

Keywords. Systems Engineering, AI, Large Language Model (LLM), Performance Benchmarking, Fine-tuning, SysMLv2, RAG, prompt engineering, chain-of-thought, MBSE.

Introduction

Generative artificial intelligence (GenAI) has shown notable promise. As an example, neural networks have been used to generate "adversarial" machine learning (ML) models to help train other ML models (e.g., (Jedrzejewski et al., 2024; Kumar et al., 2020; Pierazzi et al., 2020)). Other examples include computer vision (CV) models that are able to create images (e.g., (Esser et al., 2024; Podell et al., 2023; Rombach et al., 2022)) and videos (e.g., (Ho et al., 2022; OpenAI, 2024b)). In recent years, natural language processing (NLP), a subcategory of ML, has risen to the category of GenAI with the creation of large language models (LLMs). ChatGPT (OpenAI, 2024a), created by OpenAI, is perhaps the most notable LLM family. Other LLMs include Gemini (Team et al., 2023; Team et al., 2024), Claude (Anthropic, 2024a, 2024b), Llama (Meta, 2024), Mistral(AI, 2024a, 2024b), and Falcon (Almazrouei et al., 2023; Face, 2023; Institute, 2024; Penedo et al., 2023). Since release of a version of ChatGPT to the public, LLMs have gain thrust across many domains such as medicine (Google, 2024), critical infrastructure (Yigit et al., 2024), and the aerospace industry (Yadav, 2024).

A natural tendency is to develop means to rank the performance of the various LLMs, arising from the need conduct trades analysis in order to select LLMs that are fit-for-purpose from the rapidly increasing array of options. While many methods of evaluating LLMs quantitatively and qualitatively exist (e.g., (Gao et al., 2024; Hu & Zhou, 2024; Mizrahi et al., 2024; van Schaik & Pugh, 2024)), the "leaderboard" is established based on tasks focused on specific domains and is released on Hugging Face (Face, 2024a), a popular LLM venue. A benchmark scheme for individual domains is established through defining a set of multi-choice questions with known answers (e.g., (Deepgram, 2024; Face, 2024b; Ghazal et al., 2017; Ghazal et al., 2013; Smith & Randall, 2015; Wang et al., 2024). LLMs are prompted with the multiple-choice questions and the accuracy of the answers may be tallied to provide a relative score for each LLM, which are then ranked accordingly.

The systems engineering (SE) community has entered the realm of LLMs over roughly the last year with diverse applications and a growing set of SE specialized LLMs. Examples of LLM usage in the SE community include requirements (Bonner et al., 2024; de Oliveira et al., 2024; VanGundy, Phojanamongkolkij, et al., 2024), risk management (Bell et al., 2024), textual document generation (M Husain et al., 2024; Mohammed Husain et al., 2024), and modeling (VanGundy, Phojanamongkolkij, et al., 2024). Various LLMs have been created across the SE community, such as those created by Avian (DeHart, 2024), Ford (de Oliveira et al., 2024), NASA (VanGundy, Phojanamongkolkij, et al., 2024), Siemens (Bonner et al., 2024), and Virginia Tech (M Husain et al., 2024; Mohammed Husain et al., 2024). With the growing field, so also does the need grow for ranking LLMs in the SE domain.

The establishment of a leaderboard for SE focused LLMs has not yet occurred. However, a benchmark has been established over the last year, referred to as SysEngBench (Face, 2024b), which may be used to establish the leaderboard for the SE community. Our research leverages the SysEngBench and begins to establish the SE leaderboard of LLMs, starting with a focus on the Systems Modeling Language version 2 (SysMLv2).

LLMs are known for generating textual language, such as software code. With the transition from SysMLv1 to SysMLv2, the opportunity increases for leveraging LLMs in the SE domain and in conjunction with model-based systems engineering (MBSE). MBSE is "the formalized application of modeling to support system requirements, design, analysis, verification and validation activities beginning in the conceptual design phase and continuing throughout development and later life cycle phases (INCOSE, 2007)." SysMLv1 and SysMLv2 are MBSE modeling language versions created by the Object Management Group (OMG) and supported by many in the International Council on Systems Engineering (INCOSE) professional society. SysMLv1 is limited to graphical notation, with variations in implementation across software platforms, which adds challenges to specializing LLMs on SysMLv1. Alternatively, SysMLv2 is has both

a graphical and textual notation. The textual notation of SysMLv2 enhances the ability of the SE community to leverage LLMs.

In this article, we focus on LLMs performance in the SE domain. Due to the emphasis within the SE community on SysMLv2 and the ability to leverage LLMs in conjunction with SysMLv2, we selected to focus the impacts of various means of specializing LLMs on the SE domain through SysMLv2. Specifically, we fine-tuned a foundation model (FM) LLM on SysMLv2 as well as using RAG methods to increase the emphasis of the LLM on SysMLv2 in its output. Using SysEngBench, we compare the performance of ChatGPT-4o, the selected FM, to ChatGPT-4o instances specialized on SysMLv2 through fine-tuning and RAG. The results are unexpected and provide notable insights for the SE community to consider.

The remainder of the article progresses as follows: In the next section, we provide an overview of relevant literature. This is followed by a description and justification of our methods. We then provide documentation of the results comparing the selected LLMs. This is followed by discussion on the implications of the results. We then provide a conclusive summary of the article.

Literature review

A survey of LLMs reveals a wide array of existing LLMs. Typical foundation models (FM) are ChatGPT (OpenAI, 2024a), created by OpenAI, is perhaps the most notable LLM family. Other LLMs include Gemini (Team et al., 2023; Team et al., 2024), Claude (Anthropic, 2024a, 2024b), Llama(Meta, 2024), Mistral(AI, 2024a, 2024b), and Falcon(Almazrouei et al., 2023; Face, 2023; Institute, 2024; Penedo et al., 2023). Each have multiple versions. Newer versions may be retrained from an older version or trained from scratch. Training data varies from web-scraping to curated data sets or a combination (e.g., (Crawl, 2024)). Training a model from scratch requires high computational resources. While retraining required less computational resources, the resources required for retraining are still high and, therefore, costly. As a result, many choose to specialize existing FM through less costly methods such as fine-tuning (Minaee et al., 2024; Zhang et al., 2023), RAG (Fan et al., 2024; Li et al., 2024; Yadav, 2024), and prompt engineering (Gao, 2023; Marvin et al., 2023; Meskó, 2023; Mizrahi et al., 2024; White et al., 2023).

The use of LLMs in the SE community is still budding and, as a result, publications are largely limited to presentations from known conference venues. For example, over the last year and half, SE specialized LLM efforts may be found as presentation at targeted venues such as the Conference on Systems Engineering Research (M Husain et al., 2024), INCOSE International Symposium (Bell et al., 2024; Bonner et al., 2024; de Oliveira et al., 2024; VanGundy, Phojanamongkolkij, et al., 2024), Systems Engineering Research Center (SERC) AI workshop (Crabb et al., 2024; Demagall et al., 2023; Gadewadikar et al., 2024; Hilton & Szajnfarber, 2024; Jones, 2023; Lipizzi, 2024; Manno, 2024; Naveau, 2024; Nikam et al., 2024; Phojanamongkolkij et al., 2023; Szajnfarber et al., 2023; VanGundy, Schneide, et al., 2024; Wach et al., 2023; Wach et al., 2024), and Naval Postgraduate School Acquisition Research Symposium (Longshore et al., 2024). Example applications of LLMs for SE tasks include requirements (Bell et al., 2024; Bonner et al., 2024; VanGundy, Phojanamongkolkij, et al., 2024), risk management (Bell et al., 2024), and modeling (DeHart, 2024; Demagall et al., 2023; M Husain et al., 2024; Mohammed Husain et al., 2024; Wach et al., 2024). Methods of specializing LLM for SE include fine-tuning (Naveau, 2024; Wach et al., 2024), RAG (Manno, 2024; Phojanamongkolkij et al., 2023; VanGundy, Phojanamongkolkij, et al., 2024; VanGundy, Schneide, et al., 2024; Wach et al., 2024), and prompt engineering (Crabb et al., 2024; Longshore et al., 2024; Wach et al., 2024). With the growing use of LLM by the SE community, the need for scoring of the models increases.

While there are many means of scoring LLMs, the appropriateness of each method is dependent on the context of the task desired from the LLM. If the task is for the LLM to generate a textual description and compare the LLM generated textual description to a human generated textual description, then the MAUVE

framework (Pillutla et al., 2021) is appropriate. MAUVE assesses the similarity in probabilistic distribution between the LLM and human generated text. If the task is for the LLM to generate a textual description and compare the LLM generated textual description to an alternative LLM generated textual description, then the BERTSCORE (Zhang et al., 2019) is appropriate. Neither MAUVE nor BERTSCORE are appropriate for holistic assessment of LLM tasks focused on a specific domain. For the more holistic assessment, multiple choice questions are provided as prompts to the LLM and the LLM is then scored based on correctness established from known answers to the question. Example benchmarks include MMLU (Wang et al., 2024), Bigbench (Ghazal et al., 2017; Ghazal et al., 2013), ARC (Deepgram, 2024), and Winogrande (Code, 2024). The benchmarked scores are used to establish the leaderboard for LLM in specific domains.

The assessment of LLMs applied to the SE domain is still disorganized and a leader board does not currently exist. MAUVE has been used to assess the comparison of LLM generated text to human generated text, for example in drafting of a mission description (M Husain et al., 2024; Mohammed Husain et al., 2024; Wach et al., 2023). BERSCORE has been used to compare alternative LLM in generation of text, for example in translation of SysMLv2 code to textual description (Wach et al., 2024). To establish a leaderboard for LLM focused on SE tasks, a benchmark is necessary, which may be accomplished through using the SysEngBench (Face, 2024b).

SysEngBench may be used to establish the SE focused LLM leaderboard similarly to establishment of leaderboards for LLMs focused on other domains. SysEngBench currently covers nearly all topics in the INCOSE Handbook, which spans from foundational knowledge of systems concepts, structures, and thinking to cross-cutting methods like MBSE, to specialty engineering activities like reliability, availability, safety, and human systems integration. Therefore, we have selected to use SysEngBench as a means of assessing LLMs for this article.

Methods

In this section, we present our research question, hypothesis, and describe the means we used to answer the question. In summary, three LLMs were used that we based on ChatGPT-4o with two modified to increase specialization on SysMLv2, the LLMs were provided the questions (i.e., tasks) from SysEngBench, and the correctness of the answers were used to evaluate LLM performance.

Research Question & Hypothesis

Our selected research question was: *What are the impacts of alternative methods of specializing LLMs to the LLMs' performance on Systems Engineering tasks?*

Our hypothesis was: *Specialized LLM will perform better than foundation models (non-specialized) at Systems Engineering tasks.*

Models

Table 1 captures the LLMs that we used for the experiments as well as the methods used to specialize the LLM on SE tasks. For this limited study, we selected to use ChatGPT-4o as the foundation model, which was specialized using fine-tuning and RAG. ChatGPT is a standard name within the LLM community. ChatGPT-4o is representative of the product family. Re-training a model is expensive and of out-of-scope. Future studies will include other specialization methods, such as prompt engineering, as well as additional models, such as Gemini. We limited this study to three variants of ChatGPT-4o.

Table 1: Summary of LLM and methods of specialization

Identifier	Foundation Language Model	Model Setting(s)	Method of Specialization
Foundation model (FM)	ChatGPT-4o	Temperature = 0	None
Fine-tune model (FT)	ChatGPT-4o	Temperature = 0	Fine-tuned on SysMLv2
FT with RAG (FT-RAG)	ChatGPT-4o	Temperature = 0	Fine-tuned on SysMLv2 with provided SysMLv2 knowledge-base

For the three models: An instance of ChatGPT-4o was left unmodified for use as the control, labels as FM in Table 1. The implication is that FM would be the most generalized of the three models used for this study and should therefore perform worse at SE tasks. Two instances of ChatGPT-4o were specialized, one was fine-tuned on SysMLv2, labeled as FT in Table 1, and a second was fine-tuned as well as provided a SysMLv2 knowledge-based for RAG implementation, labeled as FT-RAG in Table 1. The implication is that FT and FT-RAG would be the less generalized and more specialized on SysMLv2, which is a modeling language created explicitly for SE, and should therefore perform better at SE tasks.

Temperature is a model setting that controls the variability of LLM responses. Because this is an initial study to establish a controlled baseline, we desired to minimize variability. Therefore, a temperature setting of zero was selected, which means the LLM responses were more deterministic (i.e., minimized variability). Future studies will vary the temperature setting.

We first optimized an instance of ChatGPT-4o using fine-tuning (FT model), which allows the user to provide a dataset of expected inputs and outputs to train an LLM's behavior on specific input. This was applied using the JSONL data storage format for input data which delimits JSON data points by new line characters. For the data, the SysMLv2 specification repository (OMG, 2024) contains code snippets covering all aspects of the syntax necessary to be used as training data for machine learning projects. The approach for this research was to take SysMLv2 diagrams that display the behavior of keywords such as a "part definition," paired with the name of the keyword as input. OpenAI requires input to apply the conversational format which matches queries to their corresponding output, so each data point contained a keyword name, such as "part definition", with matching output showing an example of that keyword in use from the SysML repository. The objective of this was to enable the LLM to associate certain keywords to diagram formats, and when given prompts related to these keywords would have further context for their usage and syntax.

To expand on the fine-tuning method, we used a dataset of ~50 chat completion examples of SysMLv2 code from the systems modeling language source repository covering a wide range of syntax from basic SysMLv2 components to entire diagrams, such as state machine diagrams. Fine-tuning was performed using OpenAI's decoder which loads the pre-trained model weights, then trains the model instance on the new dataset with a small learning rate to update model parameters. We estimate the dataset to contain ~5000-10000 tokens of pure SysMLv2 mapped to prompts describing the code behavior.

We created another instance of ChatGPT-4o using fine-tuning and RAG techniques to create a the FT-RAG model. The same fine-tuning process as the previous instance was used. FT-RAG was additionally provided with a vector-storage knowledge base, which is a database of text files that the LLM refers to for context on specific topics. For this use case, the context shown was SysMLv2 code from the example repository. The final technique performed on FT-RAG was chain-of-thought instructions, in which the LLM was instructed to provide a justification for its answers to reflect on its own choices. This was done with the goal of having the LLM check over its own work for its validity and not for human analysis.

To expand on the RAG method, we provided entire SysMLv2 diagrams as code and instructed the LLM to reference the code snippets for requests covering similar subject matter. For example, if asked to make a SysMLv2 diagram modeling a car it should reference the "Vehicle Requirements.sysml" diagram in its knowledge base. Additionally, we provided an error log that the LLM was asked to reference to ensure it did not repeat past mistakes.

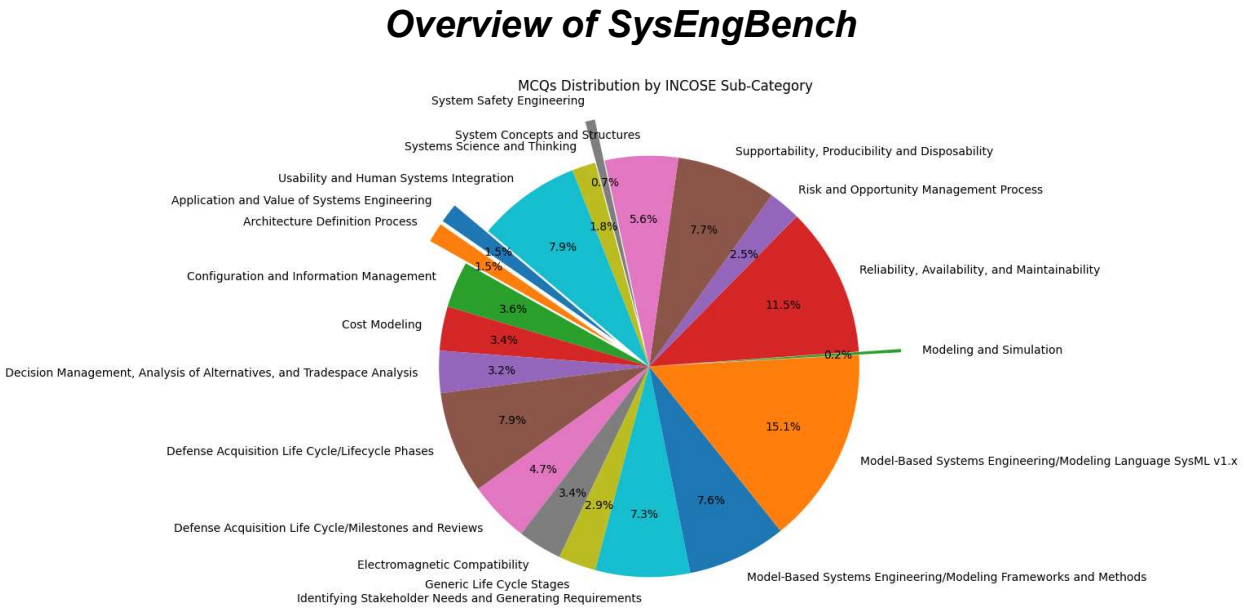


Figure 1: Distribution of SysEngBench benchmark questions

We selected to use SysEngBench (Face, 2024b) for our method of analysis. Other methods, such as MAUVE (M Husain et al., 2024; Mohammed Husain et al., 2024; Pillutla et al., 2021) and BERTSCORE (Zhang et al., 2019) as well as various existing benchmark scoring methods, were deemed to not be appropriate for our study. The desire for this research study is to assess the LLM across a broad range of SE topics and SysEngBench was created for this purpose. SysEngBench meets the needs for methodological consistency and repeatability of our study by assessing correctness in structured, discrete-response formats. The benchmark consists of 1144 multiple choice questions (MCQs). The distribution of multiple-choice questions is summarized in Figure 1 and a breakout of the Categories and Subcategories for SysEngBench are in Table 2.

Table 2: SysEngBench question major categories and subcategories

Major Categories	Subcategories	Question Count
Systems Engineering Overview	Systems Engineering Overview/System Concepts and Structures	68
	Systems Engineering Overview/Application and Value of Systems Engineering	18
	Systems Engineering Overview/Systems Science and Thinking	22
Lifecycle Stages	Lifecycle Stages/Generic Life Cycle Stages	35

	Lifecycle Stages/Defense Acquisition Life Cycle/Milestones and Reviews	57
	Lifecycle Stages/Defense Acquisition Life Cycle/Lifecycle Phases	95
Technical Processes	Technical Processes/Identifying Stakeholder Needs and Generating Requirements	88
	Technical Processes/Architecture Definition Process	18
Technical Management Processes	Technical Management Processes/Decision Management, Analysis of Alternatives, and Tradespace Analysis	39
	Technical Management Processes/ Risk and Opportunity Management Process	30
	Technical Management Processes/Configuration and Information Management	43
Cross-Cutting Systems Engineering Methods	Cross-Cutting Systems Engineering Methods/Modeling and Simulation	3
	Cross-Cutting Systems Engineering Methods/Model-Based Systems Engineering/Modeling Frameworks and Methods	92
	Cross-Cutting Systems Engineering Methods/Model-Based Systems Engineering/Modeling Language SysML v1.x	183
Specialty Engineering Activities	Specialty Engineering Activities/Cost Modeling	41
	Specialty Engineering Activities/Electromagnetic Compatibility	41
	Specialty Engineering Activities/Reliability, Availability, and Maintainability	139
	Specialty Engineering Activities/Supportability, Producibility and Disposability	93
	Specialty Engineering Activities/System Safety Engineering	9
	Specialty Engineering Activities/Usability and Human Systems Integration	96
	***Total Questions**	**1144**

Note, the total number of questions is less than the sum of each subcategory as some questions are cross-cutting and apply to multiple subcategories.

Conducting the Experiment

Each of the three models were prompted and evaluated with the questions from SysEngBench. A percentage of correctness of responses was tallied based on the overall performance, performance by major categories, and performance by sub-category.

A Python program was used to iterate through each of the SysEngBench questions in an Excel spreadsheet. The program sent questions to the LLMs using the OpenAI API. The LLMs were required to only output the MCQs answer and with no other text. The program then parsed the answer, matched it to the right answer from the spreadsheet, then made a new spreadsheet out of the results mapping: topic->question->GPT answer->real answer->correct/incorrect.

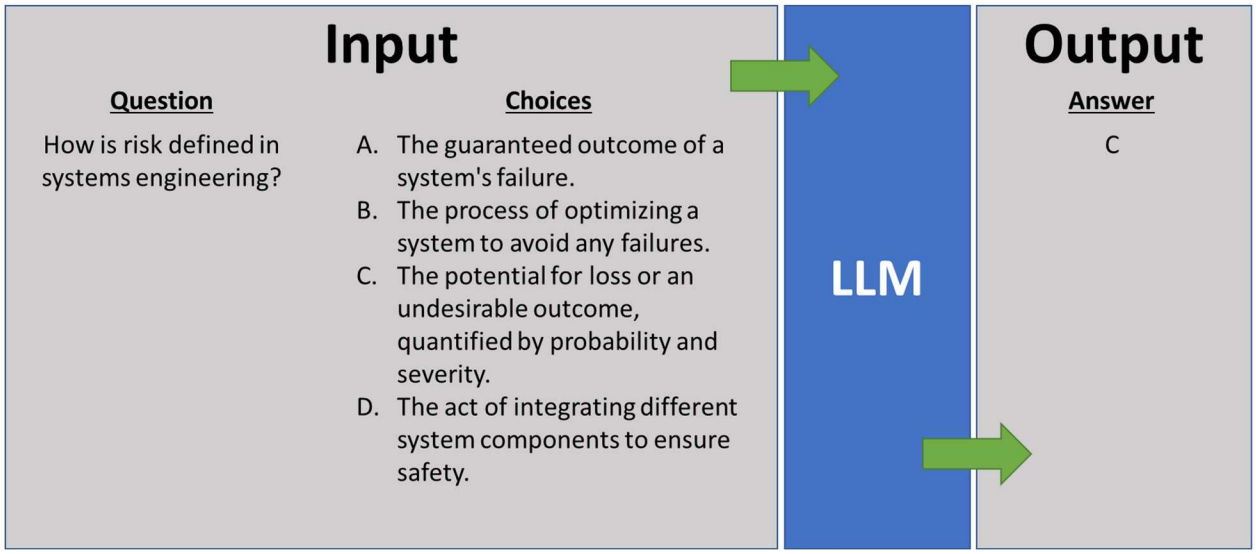


Figure 2: Example flow of MCQ as input to the LLM and output of an answer from the LLM.

Figure 2 is used to illustrate the flow of MCQ as input to the LLM and the output of the selected answer from the LLM. The answers provided by the LLM were then compared to the correct answers. The results were then tallied, categorized, and analyzed in accordance with Table 2.

The results section documents the tally. The discussion section is used to provide our interpretation of the results and potential limitations in the findings.

Results

In this section, we provide documentation of the results by overall performance, performance by major category, and performance by subcategory. The discussion section provides substantiative insights from our review of the results.

Overall Performance

The results show that FM model performed the highest overall. Furthermore, the FT-RAG model performed, while the FT model scored in the middle. It is difficult to drawing meaning at this level of detail, the difference in results is only 22 questions of 1444 between 0.96 and 0.94. Further detail is necessary to understanding the discrepancies. The overall performance results are shown in Figure 2. Insights from the results are shared in the discussion section.

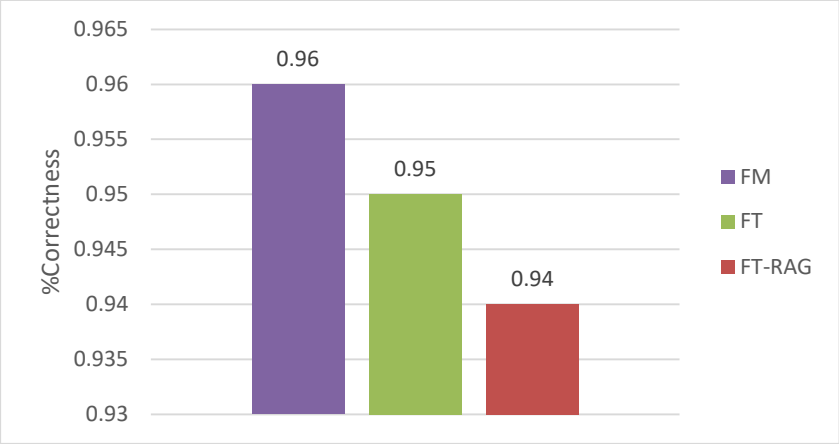


Figure 3: Overall performance of the LLM on the SysEngBench

Performance by Major Categories

The results from review of the performance by major categories supports the overall performance evaluation and shows that the FT and FT-RAG models performed worse in nearly all categories. The FT-RAG model performed worse than the other two models in nearly all categories, with the lowest score being in the category of technical management processes. The FT performed as well as the FM models in the category of specialty engineering activities. The FT and FT-RAG models performed better than the FM model in one category, which was the systems engineering overview category. The results are shown in Figure 3. Insights from the results are shared in the discussion section.

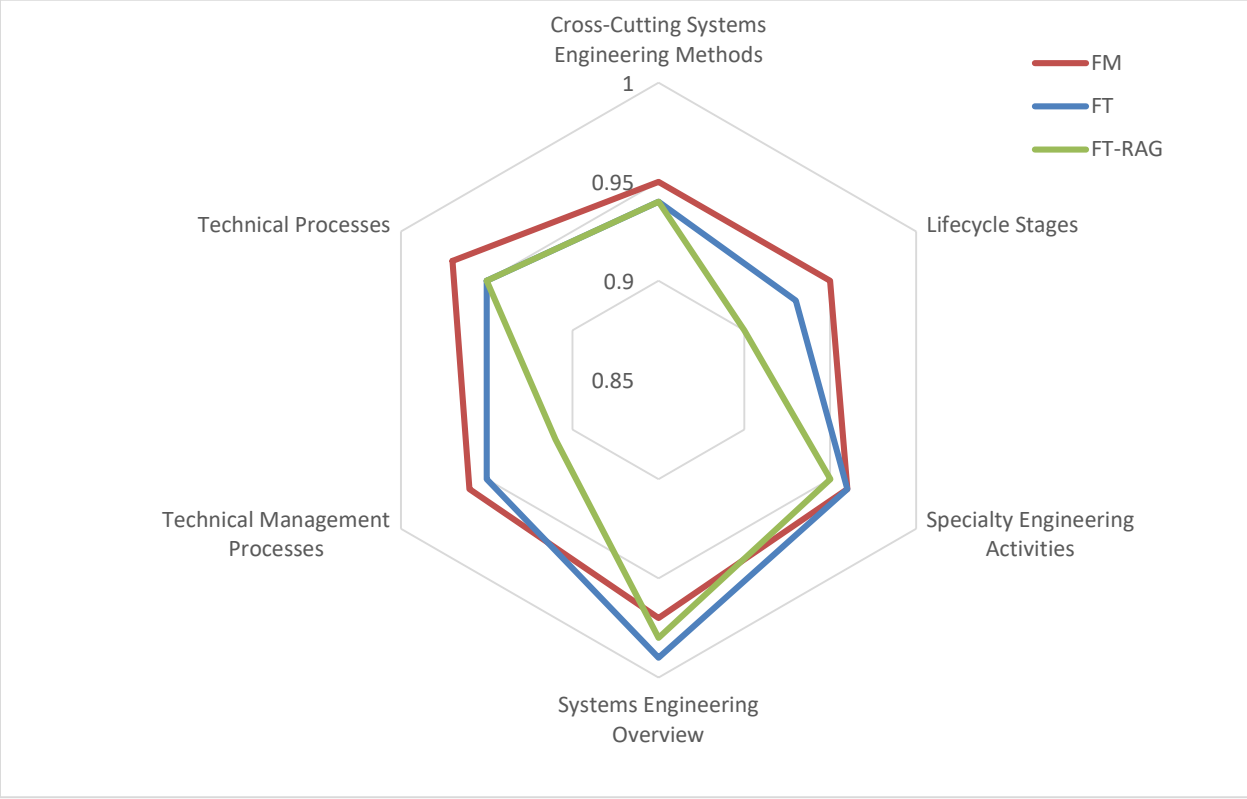


Figure 4: Performance (%Correctness) of the LLM by major category on SysEngBench

Performance by Subcategories

The results from review of the performance by subcategories supports both the overall performance and performance by major category evaluations, showing that the FT and FT-RAG models performed worse in nearly all categories. However, the distinction is less clear. The larger differentiators are on cost modeling; electromagnetic compatibility; defense acquisition lifecycle milestones and review; and decision management, analysis of alternatives, and tradespace analysis. However, the FT and FT-RAG models did not perform worse in all subcategories than the FM model. The FT and FT-RAG models performed as good or better than the FM model in subcategories systems science and thinking; system concepts and structures; application and value of systems engineering; usability and human systems integration; system safety engineering; supportability, producibility and disposability; and modeling and simulation. One potential anomaly is that the FT model out performed FM model in the subcategory of generic lifecycle stages, while the FT-RAG model under performed. Consistent with the high-level performance metrics, the FT-RAG model generally received the lowest score, with the exceptions being the subcategories model-based systems engineering/modeling frameworks and methods as well as supportability, producibility and disposability. The results are shown in Figure 4. Insights from the results are shared in the discussion section.

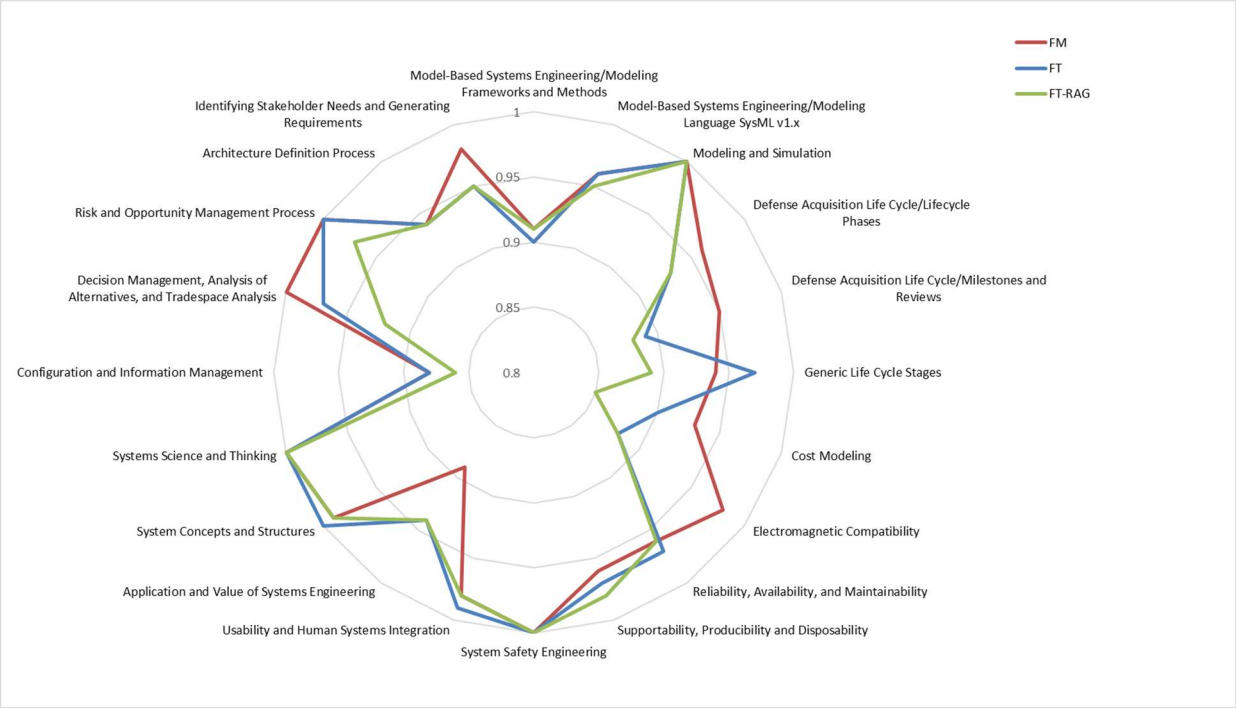


Figure 5: Performance (%Correctness) of the LLM by subcategory on SysEngBench

Discussion

Interpretation & Limitations

Overall, the interpretation of the data may seem relatively clear, which is the there is a skew showing that the specialized FT and FT-RAG models performed worse than the unmodified FM model.

The main question that should be asked here is: why would a specialized LLM perform worse than an unmodified LLM? The results show that the specialized models (FT and FT-RAG) perform worse when compared to the unmodified FM model. Reasons that a specialized model may perform worse than the

untampered model include overfitting (Xue et al., 2024; Zhang et al., 2024), dataset quality and size (Y. Liu et al., 2024; Zhang et al., 2024), loss of general knowledge (Tian et al., 2024), hyperparameter tuning (S. Liu et al., 2024), task specificity (Shi et al., 2023; van Aken, 2023), evaluation metrics (Hu & Zhou, 2024; Laskar et al., 2024), and model architecture (Moujahid et al., 2024; Raiaan et al., 2024; Rostam et al., 2024).

Overfitting generally occurs when the data is smaller and has high specificity, which our data was smaller and highly specific on SysMLv2. The result of overfitting is that the model gains specificity at the cost of generality, meaning the model may have low performance on tasks outside the specific domain and may also include variations of the domain. SE is commonly referred to and thought of as a general knowledge domain (e.g., systems science and thinking). Some overfitting on SysMLv2 may have led to the reduced performance. However, SysMLv2 was formed on the basis of being a modeling language for SE and one might expect knowledge of SysMLv2 to result in foundational knowledge of the SE domain as a whole.

Dataset quality and size pertains to representativeness and quality of the data as well as the size, which may impact the performance of the models. The size of data for SysMLv2 is certainly small. SysMLv2 is not yet formally released and, therefore, not yet fully adopted by practice, which leads to a lack of data to support specializing LLMs on SysMLv2. Data representativeness and quality, however, should not be an issue. The data used to specialize the LLMs was directly from the SysMLv2 repository (OMG, 2024), which is configuration controlled and released at intervals by the collective group developing SysMLv2. While size would contribute to the reduced performance, data representativeness and quality should be less of contributor.

Loss of general knowledge is a risk when fine-tuning LLM and may lead to "catastrophic forgetting." Essentially, the model may forget knowledge learned during pre-training. This is a possibility in our research, where the FT and FT-RAG models may have loss of general SE knowledge at the cost of gaining knowledge of SysMLv2. Exploration of loss of knowledge should be explored further.

Hyperparameter tuning is a fine-tuning method that involved, as examples, defining learning rates and batch sizes, which must be optimized appropriately. The result of inappropriate hyperparameter tuning is loss of effective learning or degradation in performance. Because we did not use hyperparameter tuning, it should not be considered a reason for decrease in performance over the FT and FT-RAG models compared to the unmodified FM model.

Task specificity issues occur when the fine-tuning dataset does not translate well to the broader context. Essentially, the fine-tuned model has knowledge that conflicts with the broader knowledge set. The implication is the SyMLv2 may conflict with the broader knowledge of SE, which the SE community may desire to explore further.

Evaluation metrics issues occur when the complexity and focus of the evaluation questions are significantly different from the training data. For our research, there may be a potential misalignment between the questions in SysEngBench and the SysMLv2 training data. SysEngBench was developed based on existing INCOSE products (e.g., (INCOSE, 2023, 2024)). SysMLv2 is being developed by systems engineers, many whom are a part of INCOSE. One might expect the two to be in alignment; however, the alignment between SysEngBench and SysMLv2 may be worth additional review.

Model architecture issues occur when the nuances from the foundation model architecture are not considered by the fine-tuning process. Additionally, model architecture issues may occur when the task environment is not suitable for the fine-tuned model. In our case, the task environment is considering to be answering multiple choice questions. The implication here is that the decrease in performance of the FT and FT-RAG models from the FM may be due to the SysMLv2 data not being aligned with answering multiple choice questions, which may be a reasonable explanation.

The SysMLv2 documents used as the data set in this study were sourced from the official SysMLv2 specification repository, which is designed primarily to illustrate the syntax and capabilities of the new textual representation of the language. While these documents serve to enhance the relevance of the study, it is important to acknowledge the limitations: the documents are relatively small in scale, not reflective of the complexity found in large-scale systems engineering projects, and were not intended for use as ML training data. Although the results demonstrate that tuning an LLM on this specific data set can reduce performance on a SE benchmark, it remains an open question whether training on different data, such as a SE textbook, would yield alternative outcomes. Nonetheless, the benchmarking contribution of this work remains valuable, and future studies should carefully document and justify data selection choices in such evaluations.

A potential limitation of this study is the inability to verify whether SysEngBench was included in the training data of ChatGPT-4o. Given that OpenAI does not disclose the full contents of its training corpus, we cannot conclusively rule out the possibility that the control model's performance may be partially attributed to data leakage rather than genuine generalization. This uncertainty presents a challenge in interpreting the results with full confidence. While addressing this limitation would require substantial effort, reproducing the experiments with an alternative LLM whose training data is known and accessible would enhance the validity and reliability of the findings.

A known limitation of the findings stems from evaluating performance differences that are relatively small, such as the delta between 0.96 and 0.94. Even with low temperature settings, LLMs can produce different outputs across repeated runs. Incorporating repeated trials and reporting variability would strengthen the interpretation of performance trends and support a more robust conclusion. We are currently in the process of conducting extended runs and analysis, which is expected to be presented at the International Symposium with a detailed journal publication to follow.

Overall, we cannot yet conclude why the models specialized on SysMLv2 performed worse than the unmodified FM model. It is reasonable that a combination of the explanations above may be possible. Further exploration is necessary. What is clear is that those looking to fine-tune LLMs should proceed with caution through awareness of the tradespace.

Considerations & Recommendations

A question for consideration by the SE community is: should SysMLv2 be reviewed and updated based on the results of this study? Perhaps, and perhaps not. The results are suggestive of a potential disconnect between SE and SysMLv2. The SysEngBench was created based on INCOSE products such as the handbook (INCOSE, 2023) and SEBoK (INCOSE, 2024), which means that the benchmark should align well with SE. SysMLv2 is a near-complete product developed by many seasoned systems engineers and INCOSE members and is being developed with the intent of capturing the essence of modeling within the context of SE. Therefore, one would expect SysMLv2 to be in alignment with SE. Nearly all of the offered explanations in the previous section present a reasonable case. Should SysMLv2 be updated? It is too early to address this question. Should SysMLv2 be reviewed in more detail within the context of this study? We certainly expect to and owe this to the SE community.

Recommendations for charting a path forward are as follows: Overall, the study should be expanded in depth and breadth. At the time of writing this article, statistical analysis has not yet been completed and is expected to be presented at the Internal Symposium with a detailed journal article to follow. Statistical analysis may provide further meaning, particularly when reviewing the differences in performance for subcategories. A simple study that may automatically adjust the results, is to adjust the temperature of the models, which would add variability of the responses. This alternative approach would necessarily be complemented with statistical analysis to assess the range in scoring of the LLMs. More SysMLv2 data, than is currently available, is necessary for refined fine-tuning of LLMs, which may change the results even in this limited context of comparison between three models. Expanding to LLMs other than ChatGPT-4o may

produce different results. Variations in responses have been noted between LLMs created by different entities and even within the ChatGPT family of LLMs. Additionally, both small language models (SLMs) and teams of models are rising in popularity and should be added to this study. SysEngBench is based on roughly 1,100 questions and an expansion of the question may also alter the results, which should be explored further. There are many directions this research may take and many conclusions that may be valuable to the SE community.

Our Next Steps

Our next include:

1. Conduct further statistical analysis to look for trends within categories and subcategories
2. Conduct additional runs on the same LLMs
3. Conduct the experiment on additional LLMs
4. Establishment of a baseline method and initial ranking for benchmarked performance of LLMs on SE tasks

Conclusion

While the main conclusion is that further research is necessary; the contribution of this article to the SE community is in the discussion which is revealing as to complexity of determine the fitness-for-purpose of LLMs for SE tasks. As an example, the results are suggestive that worth may be derived from a review of SysMLv2. Generally, it is counter-intuitive to expect that a specialized LLM would perform worse than the foundation model, ChatGPT-4o in the case of our limited study, a surprising and unexpected outcome. This reduced performance in SE tasks may result from changing the weights of the LLM to favor SysMLv2 topics more strongly, potentially to the detriment of general systems engineering tasks. This suggests that the fine-tuned model becomes more of a "subject-matter expert" (SME) rather than a "jack of all trades." While this research highlights the existence of such trade-offs, the extent of their impact needs to be further studied. In the discussion, we elaborate on many reasons why a delta in performance may exist.

References

AI, M. (2024a). *Mistral*. Retrieved Nov 27 from https://mistral.ai/

AI, M. (2024b). *Mistral on Hugging Face*. Retrieved Nov 27 from https://huggingface.co/docs/transformers/en/model_doc/mistral

Almazrouei, E., Alobeidli, H., Alshamsi, A., Cappelli, A., Cojocaru, R., Debbah, M., Goffinet, É., Hesslow, D., Launay, J., & Malartic, Q. (2023). The falcon series of open language models. *arXiv preprint arXiv:2311.16867*.

Anthropic. (2024a). *Claude*. Retrieved Nov 27 from https://claude.ai/login?returnTo=%2F%3F

Anthropic. (2024b). *Introducing Claude*. Retrieved Nov 27 from https://www.anthropic.com/news/introducing-claude

Bell, R., Longshore, R., & Madachy, R. (2024). Using AI tools for SE, from Requirements Generation and Management to Risk Identification, Analysis, and Management. INCOSE International Symposium, Dublin, Ireland.

Bonner, M., Zeller, M., Schulz, G., & Savu, A. (2024). LLM-based Approach to Automatically Establish Traceability between Requirements and MBSE. *INCOSE International Symposium*, *34*(1), 2542-2560. https://doi.org/https://doi.org/10.1002/iis2.13285

Code, P. w. (2024). *Common Sense Reasoning on WinoGrande*. Retrieved Nov 27 from https://paperswithcode.com/sota/common-sense-reasoning-on-winogrande

Crabb, E., Jones, M., & Bernard, C. (2024). Say What? Identifying the Impact of Prompt Technique on AI Generation of Systems Engineering Artifacts. AI4SE & SE4AI Workshop 2024,

Crawl, C. (2024). *Common Crawl*. Retrieved Nov 27 from https://commoncrawl.org/

de Oliveira, A. H. M., Reis, P. A., Júnior, F. S., Cavalcante, M. S., de Lima, J. V. C., Soares, L. F. C., & Marchiori, L. H. (2024). Impact Analysis of using Natural Language Processing and Large Language Model on Automated Correction of Systems Engineering Requirements. *INCOSE International Symposium*, *34*(1), 992-1007. https://doi.org/https://doi.org/10.1002/iis2.13191

Deepgram. (2024). *The ARC Benchmark: Evaluating LLMs' Reasoning Abilities*. https://deepgram.com/learn/arc-llm-benchmark-guide

DeHart, J. K. (2024). Leveraging Large Language Models for Direct Interaction with SysML v2. *INCOSE International Symposium*, *34*(1), 2168-2185. https://doi.org/https://doi.org/10.1002/iis2.13262

Demagall, A., Apaza, G., & Selva, D. (2023). *LLM Based SysML Virtual Assistant* AI4SE & SE4AI Workshop 2023, Washington, DC, USA. LLM

Texas A&M

Esser, P., Kulal, S., Blattmann, A., Entezari, R., Müller, J., Saini, H., Levi, Y., Lorenz, D., Sauer, A., & Boesel, F. (2024). Scaling rectified flow transformers for high-resolution image synthesis. *URL https://arxiv. org/abs/2403.03206*, *1*.

Face, H. (2023). *The Falcon has landed in the Hugging Face ecosystem*. Retrieved Nov 27 from https://huggingface.co/blog/falcon

Face, H. (2024a). *Hugging Face*. Retrieved Nov 27 from https://huggingface.co/

Face, H. (2024b). *SysEngBench*. https://huggingface.co/datasets/rabell/SysEngBench

Fan, W., Ding, Y., Ning, L., Wang, S., Li, H., Yin, D., Chua, T.-S., & Li, Q. (2024). A survey on rag meeting llms: Towards retrieval-augmented large language models. Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining,

Gadewadikar, J., Esho, T., & Marshall, J. (2024). *Systems Engineering Language Modeling Assistant (SELMA)* AI4SE & SE4AI Workshop 2024, Arlington, VA, USA.

Gao, A. (2023). Prompt engineering for large language models. *Available at SSRN 4504303*.

Gao, M., Hu, X., Ruan, J., Pu, X., & Wan, X. (2024). Llm-based nlg evaluation: Current status and challenges. *arXiv preprint arXiv:2402.01383*.

Ghazal, A., Ivanov, T., Kostamaa, P., Crolotte, A., Voong, R., Al-Kateb, M., Ghazal, W., & Zicari, R. V. (2017). Bigbench v2: The new and improved bigbench. 2017 IEEE 33rd International Conference on Data Engineering (ICDE),

Ghazal, A., Rabl, T., Hu, M., Raab, F., Poess, M., Crolotte, A., & Jacobsen, H.-A. (2013). Bigbench: Towards an industry standard benchmark for big data analytics. Proceedings of the 2013 ACM SIGMOD international conference on Management of data,

Google. (2024). *Med-PaLM*. Retrieved Nov 27 from https://sites.research.google/med-palm/

Hilton, S., & Szajnfarber, Z. (2024). Towards a Work Systems View of Human AI Collaboration in Systems Engineering and Design: The Case of Conceptual Design with a ChatGPT Partner. AI4SE & SE4AI Workshop 2024, Arlington, VA, USA.

Ho, J., Chan, W., Saharia, C., Whang, J., Gao, R., Gritsenko, A., Kingma, D. P., Poole, B., Norouzi, M., & Fleet, D. J. (2022). Imagen video: High definition video generation with diffusion models. *arXiv preprint arXiv:2210.02303*.

Hu, T., & Zhou, X.-H. (2024). Unveiling LLM Evaluation Focused on Metrics: Challenges and Solutions. *arXiv preprint arXiv:2404.09135*.

Husain, M., Topcu, T., & Wach, P. (2024). Can Large Language Models Accelerate Digital Transformation by Generating Expert-Like Systems Engineering Artifacts? Insights from an Empirical Exploration. Conference on Systems Engineering Research, Tucson, AZ, USA.

Husain, M., Wach, P., & Topcu, T. G. (2024). Expert-Like Systems Engineering Artifacts? Insights from an Empirical. The Proceedings of the 2024 Conference on Systems Engineering Research,

INCOSE. (2007). *Systems Engineering Vision 2020*.

INCOSE. (2023). *INCOSE Systems Engineering Handbook* (5th ed.).

INCOSE. (2024). *Guide to the Systems Engineering Body of Knowledge (SEBoK)*. Retrieved Nov 27 from https://sebokwiki.org/wiki/Guide_to_the_Systems_Engineering_Body_of_Knowledge_(SEBoK)

Institute, T. I. (2024). *Introducing Falcon Mamba 7B*. Retrieved Nov 27 from https://falconllm.tii.ae/

Jedrzejewski, F. V., Thode, L., Fischbach, J., Gorschek, T., Mendez, D., & Lavesson, N. (2024). Adversarial machine learning in industry: A systematic literature review. *Computers & Security*, 103988.

Jones, M. (2023). Generating User-centric Intelligent Designs for Digital
Engineering (GUIDE). AI4SE & SE4AI Workshop 2023, Washington, DC, USA.

Kumar, R. S. S., Nyström, M., Lambert, J., Marshall, A., Goertzel, M., Comissoneru, A., Swann, M., & Xia, S. (2020). Adversarial machine learning-industry perspectives. 2020 IEEE security and privacy workshops (SPW),

Laskar, M. T. R., Alqahtani, S., Bari, M. S., Rahman, M., Khan, M. A. M., Khan, H., Jahan, I., Bhuiyan, A., Tan, C. W., & Parvez, M. R. (2024). A systematic survey and critical review on evaluating large language models: Challenges, limitations, and recommendations. Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing,

Li, J., Yuan, Y., & Zhang, Z. (2024). Enhancing llm factual accuracy with rag to counter hallucinations: A case study on domain-specific queries in private knowledge-bases. *arXiv preprint arXiv:2403.10446*.

Lipizzi, C. (2024). Large Language Models in Enhanced Electronic Warfare: Applications, Benefits, Limitations and Future Directions. AI4SE & SE4AI Workshop 2024, Arlington, VA, USA.

Liu, S., Gao, C., & Li, Y. (2024). Large Language Model Agent for Hyper-Parameter Optimization. *arXiv preprint arXiv:2402.01881*.

Liu, Y., Tao, S., Zhao, X., Zhu, M., Ma, W., Zhu, J., Su, C., Hou, Y., Zhang, M., & Zhang, M. (2024). Coachlm: Automatic instruction revisions improve the data quality in llm instruction tuning. 2024 IEEE 40th International Conference on Data Engineering (ICDE),

Longshore, R., Bell, R., & Madachy, R. (2024). Leveraging Generative AI to Build, Modify, and Query MBSE Models. NPS Acquisition Research Symposium 2024, Monterey, CA, USA.

Manno, N. (2024). Accelerating Semantic Digital Thread User Queries Using LLMs. AI4SE & SE4AI Workshop 2024,

Marvin, G., Hellen, N., Jjingo, D., & Nakatumba-Nabende, J. (2023). Prompt engineering in large language models. International conference on data intelligence and cognitive informatics,

Meskó, B. (2023). Prompt engineering as an important emerging skill for medical professionals: tutorial. *Journal of medical Internet research*, *25*, e50638.

Meta. (2024). *Introducing Llama 3.2*. Retrieved Nov 27 from https://www.llama.com/

Minaee, S., Mikolov, T., Nikzad, N., Chenaghlu, M., Socher, R., Amatriain, X., & Gao, J. (2024). Large language models: A survey. *arXiv preprint arXiv:2402.06196*.

Mizrahi, M., Kaplan, G., Malkin, D., Dror, R., Shahaf, D., & Stanovsky, G. (2024). State of what art? a call for multi-prompt llm evaluation. *Transactions of the Association for Computational Linguistics*, *12*, 933-949.

Moujahid, H., Boutahar, K., El Gannour, O., Saleh, S., Cherradi, B., & El Abbassi, A. (2024). A Scoping Review of Large Language Models: Architecture and Applications. 2024 4th International Conference on Innovative Research in Applied Science, Engineering and Technology (IRASET),

Naveau, M. (2024). LLM Co-pilots for Domain Specific Modeling Languages. AI4SE & SE4AI Workshop 2024, Arlington, VA, USA.

Nikam, A., McCollum, J., Vazirani, V., & Chhun, C. (2024). The Use of Generative GenAI to Unlock Value from Defense ERP Business Systems. AI4SE & SE4AI Workshop 2024, Arlington, VA, USA.

OMG. (2024). *OMG Systems Modeling Language™ (SysML®) v2 Release*. Retrieved Nov 27 from https://github.com/Systems-Modeling/SysML-v2-Release

OpenAI. (2024a). *OpenAI ChatGPT*. Retrieved Nov 27 from https://openai.com/

OpenAI. (2024b). *Video generation models as world simulators*. Retrieved Nov 27 from https://openai.com/index/video-generation-models-as-world-simulators/

Penedo, G., Malartic, Q., Hesslow, D., Cojocaru, R., Cappelli, A., Alobeidli, H., Pannier, B., Almazrouei, E., & Launay, J. (2023). The RefinedWeb dataset for Falcon LLM: outperforming curated corpora with web data, and web data only. *arXiv preprint arXiv:2306.01116*.

Phojanamongkolkij, N., VanGundy, B., Polavarapu, R., Levitt, I., & Brown, B. (2023). Requirement Discovery Using Embedded Knowledge Graph with ChatGPT. AI4SE & SE4AI Workshop 2023, Washington, DC, USA.

Pierazzi, F., Pendlebury, F., Cortellazzi, J., & Cavallaro, L. (2020). Intriguing properties of adversarial ml attacks in the problem space. 2020 IEEE symposium on security and privacy (SP),

Pillutla, K., Swayamdipta, S., Zellers, R., Thickstun, J., Welleck, S., Choi, Y., & Harchaoui, Z. (2021). Mauve: Measuring the gap between neural text and human text using divergence frontiers. *Advances in Neural Information Processing Systems*, *34*, 4816-4828.

Podell, D., English, Z., Lacey, K., Blattmann, A., Dockhorn, T., Müller, J., Penna, J., & Rombach, R. (2023). Sdxl: Improving latent diffusion models for high-resolution image synthesis. *arXiv preprint arXiv:2307.01952*.

Raiaan, M. A. K., Mukta, M. S. H., Fatema, K., Fahad, N. M., Sakib, S., Mim, M. M. J., Ahmad, J., Ali, M. E., & Azam, S. (2024). A review on large Language Models: Architectures, applications, taxonomies, open issues and challenges. *IEEE Access*.

Rombach, R., Blattmann, A., Lorenz, D., Esser, P., & Ommer, B. (2022). High-resolution image synthesis with latent diffusion models. Proceedings of the IEEE/CVF conference on computer vision and pattern recognition,

Rostam, Z. R. K., Szénási, S., & Kertész, G. (2024). Achieving Peak Performance for Large Language Models: A Systematic Review. *IEEE Access*.

Shi, C., Su, Y., Yang, C., Yang, Y., & Cai, D. (2023). Specialist or generalist? instruction tuning for specific NLP tasks. *arXiv preprint arXiv:2310.15326*.

Smith, W. A., & Randall, R. B. (2015). Rolling element bearing diagnostics using the Case Western Reserve University data: A benchmark study. *Mechanical systems and signal processing*, *64*, 100-131.

Szajnfarber, Z., Adeyeye, O., & Pless, R. (2023). *Opportunities and Risks of Incorporating LLMs in the Systems Engineering and Design Workflow: A Case Study of Robotic System Design Process* AI4SE & SE4AI Workshop 2023, Washington, DC, USA.

Team, G., Anil, R., Borgeaud, S., Alayrac, J.-B., Yu, J., Soricut, R., Schalkwyk, J., Dai, A. M., Hauth, A., & Millican, K. (2023). Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.

Team, G., Georgiev, P., Lei, V. I., Burnell, R., Bai, L., Gulati, A., Tanzer, G., Vincent, D., Pan, Z., & Wang, S. (2024). Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*.

Tian, B., Liang, X., Cheng, S., Liu, Q., Wang, M., Sui, D., Chen, X., Chen, H., & Zhang, N. (2024). To forget or not? towards practical knowledge unlearning for large language models. *arXiv preprint arXiv:2407.01920*.

van Aken, B. (2023). Exploration and adaptation of large language models for specialized domains.

van Schaik, T. A., & Pugh, B. (2024). A field guide to automatic evaluation of llm-generated summaries. Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval,

VanGundy, B., Phojanamongkolkij, N., Brown, B., Polavarapu, R., & Bonner, J. (2024). Requirement Discovery Using Embedded Knowledge Graph with ChatGPT. *INCOSE International Symposium*, *34*(1), 2011-2027. https://doi.org/https://doi.org/10.1002/iis2.13253

VanGundy, B., Schneide, M., Phojanamongkolkij, N., Levitt, I., & Brown, B. (2024). Developing Concepts of Operations Using Multi-Step Tool Techniques with Large Language Models. AI4SE & SE4AI Workshop 2024, Arlington, VA, USA.

Wach, P., Husain, M., & Topcu, T. G. (2023). Large Language Model Enabled Generation of Systems Engineering Artifacts. AI4SE & SE4AI Workshop 2023, Washington, DC, USA.

Wach, P., Jugan, B., & Lucero, S. (2024). Using Large Language Models to Accelerate Development of Complex Systems. AI4SE & SE4AI Workshop 2024, Arlington, VA, USA.

Wang, Y., Ma, X., Zhang, G., Ni, Y., Chandra, A., Guo, S., Ren, W., Arulraj, A., He, X., & Jiang, Z. (2024). Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. *arXiv preprint arXiv:2406.01574*.

White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J., & Schmidt, D. C. (2023). A prompt pattern catalog to enhance prompt engineering with chatgpt. *arXiv preprint arXiv:2302.11382*.

Xue, F., Fu, Y., Zhou, W., Zheng, Z., & You, Y. (2024). To repeat or not to repeat: Insights from scaling llm under token-crisis. *Advances in Neural Information Processing Systems*, *36*.

Yadav, S. (2024). AeroQuery RAG and LLM for Aerospace Query in Designs, Development, Standards, Certifications. 2024 IEEE International Conference on Electronics, Computing and Communication Technologies (CONECCT),

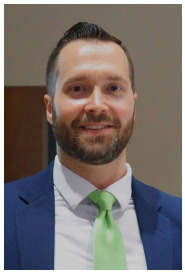
Yigit, Y., Ferrag, M. A., Sarker, I. H., Maglaras, L. A., Chrysoulas, C., Moradpoor, N., & Janicke, H. (2024). Critical infrastructure protection: Generative ai, challenges, and opportunities. *arXiv preprint arXiv:2405.04874*.

Zhang, B., Liu, Z., Cherry, C., & Firat, O. (2024). When scaling meets llm finetuning: The effect of data, model and finetuning method. *arXiv preprint arXiv:2402.17193*.

Zhang, S., Dong, L., Li, X., Zhang, S., Sun, X., Wang, S., Li, J., Hu, R., Zhang, T., & Wu, F. (2023). Instruction tuning for large language models: A survey. *arXiv preprint arXiv:2308.10792*.

Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., & Artzi, Y. (2019). Bertscore: Evaluating text generation with bert. *arXiv preprint arXiv:1904.09675*.

Biography



Paul Wach, Ph.D. Dr. Paul Wach is a Research Assistant Faculty with the Intelligent Systems Division of the Virginia Tech National Security Institute. His research interests include the intersection of theoretical foundations of systems engineering, digital transformation, and artificial intelligence. His research is motivated by the need to produce practical theory to serve as the scientific foundations for engineering systems within the evolving digital paradigm. He is currently serving INCOSE as the Assistant Director for Student Matter, under the Academic Matters directorate. In this role, He oversees the INCOSE Student Divisions as well as the Systems Engineering and Architecting Doctoral Student Network (SEANET). He also teaches a senior undergraduate and graduate level course on digital engineering (DE) at Virginia Tech, through the Industrial & Systems Engineering department. Prior to Virginia Tech, he was a lead for enterprise digital transformation with The Aerospace Corporation. Other work experience includes the Department of Energy, where he served in lead engineering and management roles ranging in magnitude from $1-12B as well as work experience with two National Laboratories; and the medical industry, where he worked on cutting edge technology such as an artificial kidney. He received a B.S. in Biomedical Engineering from Georgia Tech, M.S. in Mechanical Engineering from the University of South Carolina, and Ph.D. in Industrial & Systems Engineering from Virginia Tech.

Ryan Bell. Ryan Bell is an 9 year experienced engineer in the defense industry. In his current role at Naval Information Warfare Center Atlantic (NIWC LANT), Ryan provides modeling and simulation expertise to a variety of programs for the Navy and USMC. He specializes in simulating communication systems in complex environments and is an advocate for the use of digital engineering early in the systems engineering lifecycle. Ryan earned a BS in Electrical Engineering from Clemson University, a MS in Electrical Engineering from Clemson University with a focus on Electronics, and is currently pursuing his PhD in Systems Engineering at the Naval Postgraduate School. He is a South Carolina registered Professional Engineer (PE), published author, and teacher.

Brady Jugan. Brady Jugan is a rising senior in computer science at Virginia Tech and is currently an RTX Fellow. Mr. Jugan enjoys research on the topic of generative AI and has created verification & validation methods for large language models (LLMs) to ensure that the output is fit-for-purpose. An example is automating the generation for systems modeling language version 2 (SysMLv2) models for various domains. His work is helping to shape the Systems Simulated Training Environment for Digital Engineering (STEDE) at the Virginia Tech National Security Institute.

Ryan Longshore. Ryan Longshore is a 20-year veteran of the defense and electric utility industries. At Naval Information Warfare Center Atlantic, he leads teams developing and integrating new technologies into Navy command centers. He is involved in the Navy's digital engineering transformation with a focus on model-based systems and model-based engineering. Ryan holds a BS in Electrical Engineering (Clemson), a MS in Systems Engineering (SMU), and is pursuing a PhD is Systems Engineering at the Naval Postgraduate School. He is a South Carolina Registered Professional Engineer, an INCOSE CSEP, and holds the OMG SysML Model Builder Fundamental Certification.

Raymond Madachy. Raymond Madachy, Ph.D., is a Professor in the Systems Engineering Department at the Naval Postgraduate School. His research interests include systems engineering tool environments for digital engineering, modeling and simulation of systems and software engineering processes, generative AI, and system cost modeling. He has developed cost estimation tools for systems and software engineering, and created the Systems Engineering Library (*se-lib*). His books include *Software Process Dynamics*, *What Every Engineer Should Know about Modeling and Simulation*, *What Every Engineer Should Know about Python; a*nd co-authored *Software Cost Estimation with COCOMO II* and *Software Cost Estimation Metrics Manual for Defense Systems*.